CONNECT THE DOTS…

An Interactive Learning Activity for Children

Name	**Cheryl Pinto**
Email	cp16november@gmail.com
GitHub	https://github.com/cherylpinto
Prototype Repository GitHub	https://github.com/cherylpinto/GSOC-2026
University	K.J. Somaiya College of Engineering, Vidyavihar, Mumbai, Maharashtra, India
Program	B.Tech Computer Engineering
Timezone	Indian Standard Time (UTC+5:30)
Project Size:	Medium
Organization	Sugarizer, Sugar Labs
Organization Mentor	Lionel Laske

PERSONAL OVERVIEW

I am a Computer Science Engineering student at KJSCE, Mumbai with a strong focus on designing intuitive and impactful user experiences.

I have previously contributed to other Sugar Labs codebases and am now looking to extend my contributions to Sugarizer. I have taken the time to understand its architecture and workflows, and have already implemented and explored activity-level features within the platform.

I am motivated to contribute to open source in a way that creates real educational impact and I look forward to building thoughtful and engaging improvements in Sugarizer through GSoC.

PREVIOUS CONTRIBUTIONS

I have contributed to the Sugar Labs ecosystem through the **www-v2 repository**, focusing on UI/UX improvements, dark mode consistency, and feature enhancements. While I was unable to contribute directly to Sugarizer, I have independently explored its codebase and built prototypes to understand activity development in depth.

I also worked on the *Pawn activity* to understand how to build a complete Sugarizer activity from scratch, including Journal integration, localization, presence (multi-user), palettes, and tutorial integration.

Pull Requests:

- https://github.com/sugarlabs/www-v2/pull/638
  → Enhanced hover feedback for Developer Resources cards in dark mode by adding an accent highlight, improving interactivity without altering layout or behavior.
- https://github.com/sugarlabs/www-v2/pull/628
  → Fixed dark mode text visibility issue in the "Our Strategy" mobile view by correcting heading color styles, improving readability and consistency.
- https://github.com/sugarlabs/www-v2/pull/614
  → Implemented comprehensive dark mode support for Developer Links by adding proper color variants for backgrounds, text, and accents, improving contrast and theme consistency.
- https://github.com/sugarlabs/www-v2/pull/471
  → Redesigned the dark mode toggle with animated transitions, improved accessibility (ARIA), and enhanced visual feedback using modern UI elements.
- https://github.com/sugarlabs/www-v2/pull/467
  → Designed and implemented a fully responsive, dark-mode-compatible Contact Us page with animations, structured contact sections, and integrated community links

PROJECT OVERVIEW

This project aims to create an engaging and interactive **Connect the Dots activity** that helps children learn through play.

Children will be able to connect dots to form shapes, letters and objects, encouraging pattern recognition, sequencing and creativity. The activity will offer different ways to play and learn:

- A **free drawing mode** where children can connect dots creatively and explore their own ideas.
- A **guided mode** where children connect dots in order, helping them learn numbers and sequences.
- A **game mode** that introduces simple challenges and interaction to make learning more fun.

To further enhance the experience:

- Children can choose from different templates like shapes, animals and letters.
- They can also **create their own dot patterns**, encouraging creativity and ownership.
- Progress can be saved so they can continue later.
- Collaborative features will allow shared interaction in a playful environment.

The goal is to make learning intuitive, enjoyable and interactive by combining structured activities with creative freedom.

IMPACT ON SUGARIZER

Sugarizer is a web-based reimplementation of the Sugar learning platform, originally designed for the One Laptop per Child (OLPC) initiative. It aims to provide children with an accessible, interactive, and activity-based learning environment that encourages exploration, creativity and collaboration. By bringing Sugar to modern web and mobile platforms, Sugarizer makes these educational experiences widely available.

My proposed **Connect the Dots activity** aligns closely with Sugarizer's philosophy of *learning through doing*. It introduces an engaging way for children to develop essential cognitive skills such as pattern recognition, sequencing and spatial understanding through interactive play.

This activity will contribute to Sugarizer by:

- Introducing a multi-mode activity combining creativity, guidance and gameplay.
- Strengthening support for early learning concepts like numbers, shapes, and patterns.
- Enabling template-based and user-created content for flexible learning.
- Encouraging collaborative and shared learning experiences.
- Providing an extensible structure for future educational activities.

TECHNOLOGIES USED

❖ Technologies:

- **JavaScript (VanillaJS)** for application logic, state management and interaction handling.
- **HTML5** and **CSS3** for activity structure, styling, and responsive UI.
- **HTML5 Canvas API** for rendering dots, lines, fills, and interactive visuals.
- **JSON** for template definitions and state persistence.

❖ Frameworks & Platform Knowledge:

- **Sugar Web Framework APIs** (Activity lifecycle, Datastore/Journal, Presence for collaboration, Palette components).
- RequireJS (AMD modules) for modular architecture and dependency management.
- Understanding of Sugarizer environment and activity structure.

❖ Relevant Skills:

- Strong understanding of Sugarizer architecture, activity structure and workflows.
- UI/UX-focused development for interactive applications.
- Git/GitHub for version control and collaboration.
- Managing asynchronous operations and user-driven interactions.
- End-to-end feature development through prototype implementation.

PROJECT IDEA

The proposed project aims to develop an interactive **Connect the Dots activity** for Sugarizer that combines **creativity, guided learning, and gameplay** within a single experience. The activity is designed to be **intuitive and engaging** for children, allowing them to explore patterns, sequences, and shapes through simple interactions.

It will support **multiple modes of interaction**, enabling both structured learning and free-form creativity, while also integrating key Sugarizer features such as collaboration and persistence.

All modes are built on a common dot-grid interaction and filling system derived from Draw Mode. The interface is kept minimal to ensure a distraction-free experience for children.

The following sections describe the design and technical implementation of the activity:

1. DIRECTORY STRUCTURE CURRENTLY:

```
ConnectTheDots.activity/
│
├── index.html                # Activity entry point (UI layout, toolbar)
│
├── activity/                 # Sugar activity metadata
│   ├── activity.info         # Activity registration and configuration
│   └── activity-icon.svg     # Activity icon
│
├── js/                       # Core logic and interaction
│   ├── game.js               # Main controller (modes, undo/redo, state)
│   ├── canvas.js             # Rendering and user interaction handling
│   ├── line.js               # Line management and graph logic
│   ├── templates.js          # Built-in and custom templates
│   │
│   └── modes/                # Mode-specific implementations
│       ├── drawMode.js       # Free drawing mode
│       ├── numberMode.js     # Guided numbered mode
│       └── captureMode.js    # Game (capture) mode
│
├── css/                      # Styling
│   └── activity.css          # Main styles for UI and canvas
│
├── icons/                    # SVG icons for toolbar and UI
│
└── lib/                      # External libraries and helpers
    ├── require.js            # Module loader
    ├── sugar-web/            # Sugarizer framework utilities
    ├── i18n/                 # Localization support
    └── ...                   # Other helper libraries
```

2. TOOLBAR STRUCTURE:

The activity is organized around a **top navigation toolbar** that provides a clear and intuitive interface for children to interact with different features.

The toolbar is divided into functional sections arranged from left to right:

- **Activity Identity:** Displays the *Connect the Dots* activity icon, maintaining consistency with Sugarizer's activity-based interface.
- **Mode Selection:** Three primary modes - *Draw*, *Number*, and *Capture*, are accessible through dedicated buttons. Switching modes updates the active state and dynamically adjusts available tools.
- **Color Control:** A color picker allows users to select line and fill colors, supporting creative interaction while keeping the interface minimal.
- **Collaboration:** A network sharing button enables multi-user interaction through Sugarizer's Presence system.

- **Editing Tools:** Undo, Redo, and Clear actions allow users to safely explore and correct their work without penalty.
- **Persistence Actions:** The Capture Icon saves the current canvas as a PNG image to the Journal.
- **Help and Exit Controls:** Help (tutorial) and Stop buttons are positioned on the right for easy access, following Sugarizer UI conventions.

Figure 1: Toolbar

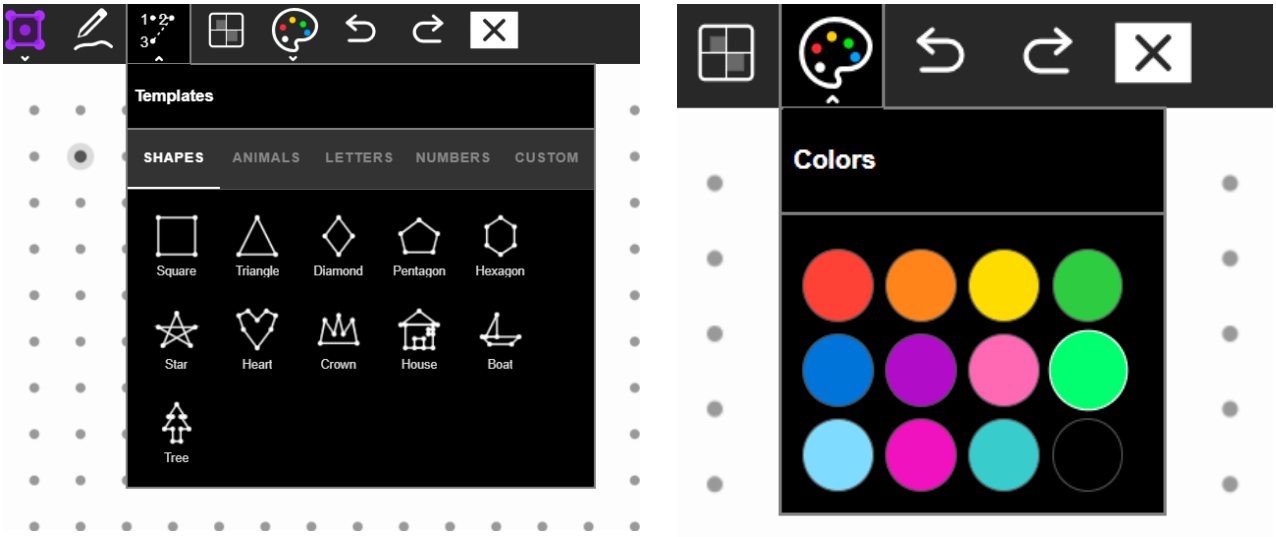


Figure 2: Dropdowns

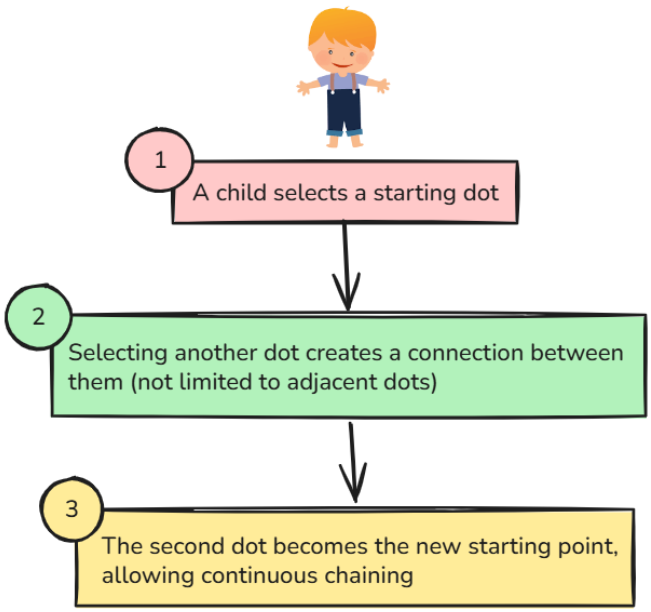
3. DRAW MODE:

Draw Mode is designed as an intuitive yet **structured interaction system** where children can create shapes by connecting dots. Instead of freehand drawing, the mode uses a **grid-based graph approach** to ensure consistency and meaningful interaction.

❖ Core Design Approach

- Click-to-connect interaction between any two dots
- Grid-constrained drawing for consistency
- Automatic detection and filling of closed regions
- State-driven system supporting undo/redo and persistence

❖ Interaction Flow



❖ Grid and Line Handling

- Dots are placed on a fixed grid and input snaps to nearest valid points
- Long connections are decomposed using **Bresenham logic** to ensure continuous paths

❖ Graph and Cycle Detection

- Dots act as nodes and connections as undirected edges
- **BFS traversal** is used to check if a path already exists between two dots
- If a path exists, the new edge forms a closed loop (polygon)

❖ Filling and Rendering

- Closed regions are filled using a **BFS-based flood fill algorithm**
- Offscreen rendering is used to avoid visual artifacts

❖ Visual Feedback and Interaction States

- The system provides continuous feedback:
  - Highlighted dots on hover
  - Clear indication of selected points
  - Suppression of dots inside filled regions for visual clarity

❖ Color Integration

- Draw Mode supports dynamic color selection:
  - Colors are selected through the toolbar palette
  - Both lines and filled regions reflect the chosen color
  - Changes are applied in real time

Figure 3: Draw Mode Canvas

❖ State and Integration

- Actions are stored as state snapshots for undo/redo
- State is serialized for **Journal persistence**
- Supports **real-time collaboration** using Presence APIs, synchronizing drawing actions across users

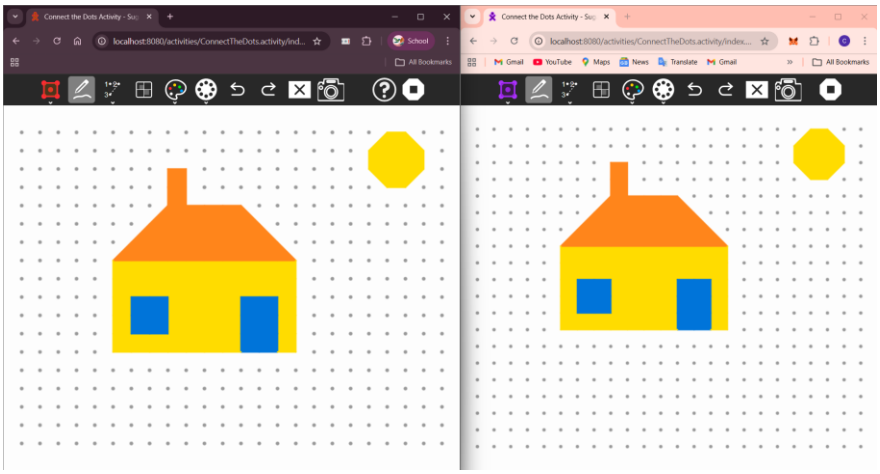


Figure 4: Multiplayer View

❖ Responsiveness and Interaction (Shown in the video)

- Designed to work across **different screen sizes and devices**
- Grid and canvas dynamically adapt to available space
- Supports both **mouse and touch interactions**, ensuring usability on tablets and mobile devices

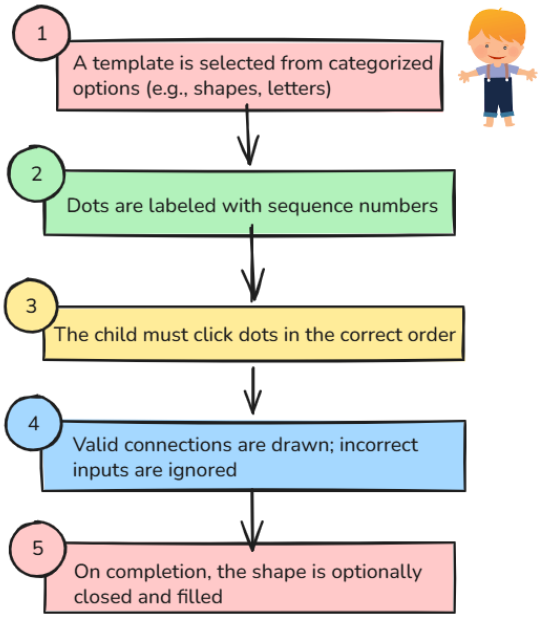
4. NUMBER MODE

Number Mode introduces a **guided connect-the-dots experience** where children complete shapes by following a predefined sequence. Unlike Draw Mode, interactions are **validated and ordered**, helping learners understand sequencing and structure.

❖ Core Design

- Enforces **step-by-step dot connection** based on a predefined order
- Supports both **open and closed shapes** (e.g., letters vs polygons)
- Uses the same **grid and fill system as Draw Mode** for consistency
- Supports both predefined templates and user-authored custom templates

❖ Interaction Flow



❖ Template System

- Templates are organized into categories like **Shapes, Animals, Letters, and Numbers**
- Each template consists of one or more **parts (dot sequences)**
- Supports:
  - **Open paths** for letters and strokes
  - **Closed paths** for shapes with automatic filling
- Templates are normalized to fit the grid dynamically

❖ Sequential Validation Logic

- Each click is matched against the **expected next index**
- Only correct inputs are accepted, enforcing structured learning
- Connections are drawn deterministically to preserve template shape

❖ Custom Template Authoring

Number Mode also supports **user-generated templates**:

- Users can create their own dot sequences in an authoring mode
- Each click adds a labeled point and builds a path

- On completion:
  - Coordinates are normalized relative to the grid
  - Template is saved and added to a custom category
- Enables creativity alongside guided learning

State and Integration

- Integrated with the central game system for **mode switching and state tracking**
- Supports **undo/redo via state snapshots**
- Custom templates and progress are stored using **JSON serialization**
- Compatible with **Journal persistence and collaboration features**

Templates

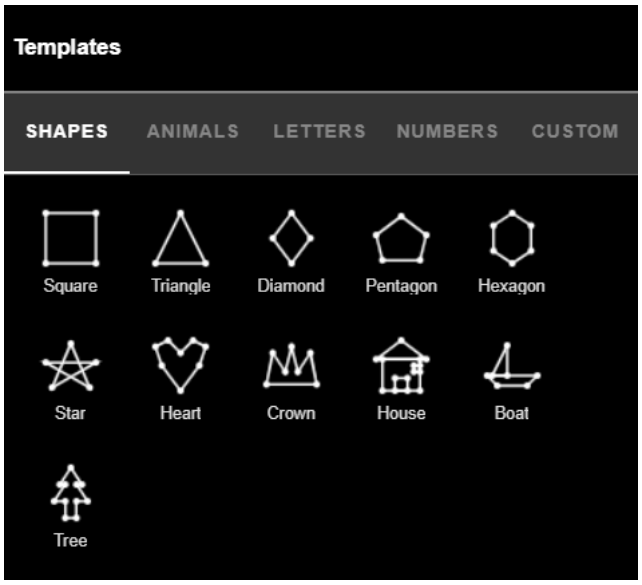


Figure 5: Shape Templates

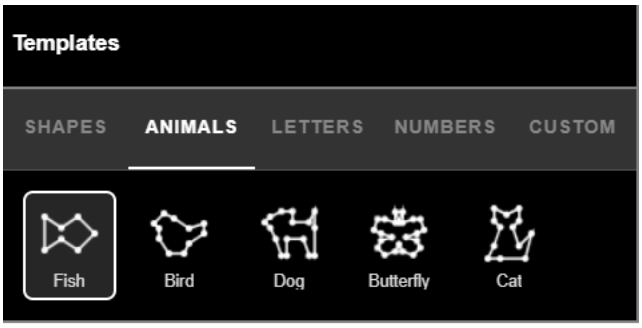


Figure 6: Animal Templates

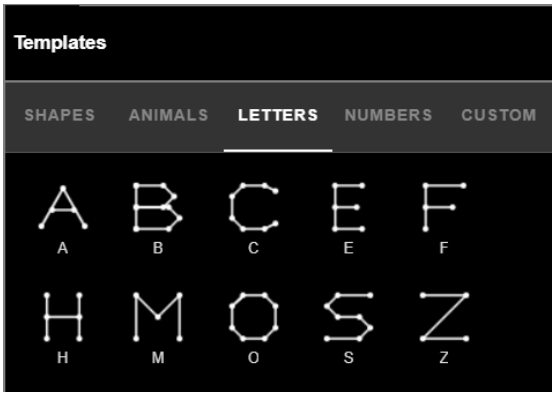


Figure 7: Letter Templates

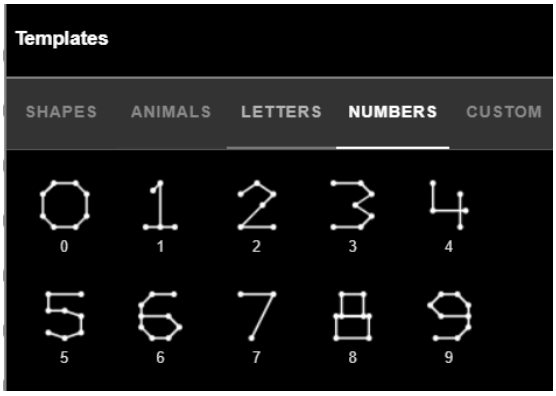


Figure 8: Number Templates

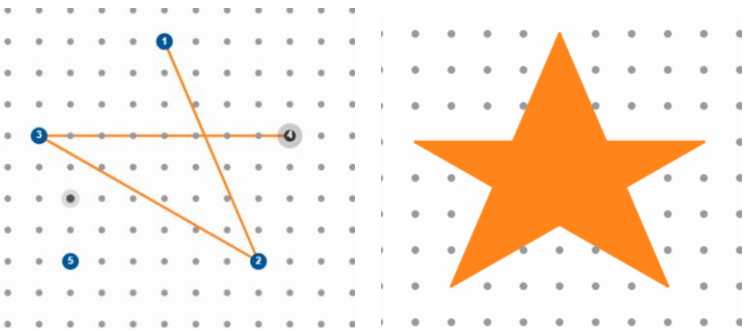


Figure 9: Number mode for Shapes

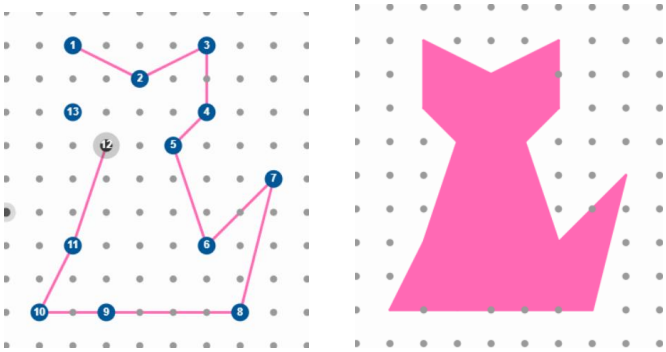


Figure 10: Number mode for animals

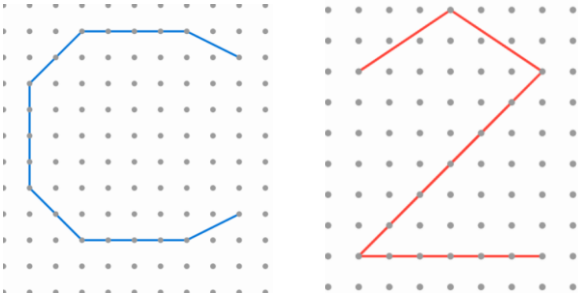


Figure 11: Number mode for letters and numbers

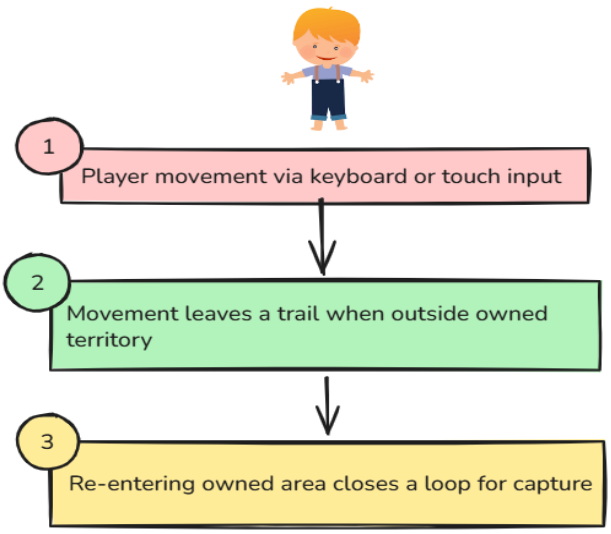
5. GAME MODE

Game Mode introduces a competitive **child vs AI gameplay system** where players capture territory on the grid through movement and enclosure.

❖ Core Design

- Real-time grid-based gameplay with territory capture mechanics
- Player vs AI interaction with independent movement and state
- Time-based game loop with scoring and win conditions

❖ Interaction and Controls



❖ Territory Capture Logic

- Enclosed regions are detected and filled using a **BFS-based flood fill algorithm**
- Captured area is assigned to the player and contributes to score
- Opponent trails inside captured regions are eliminated

❖ Collision and Game Rules

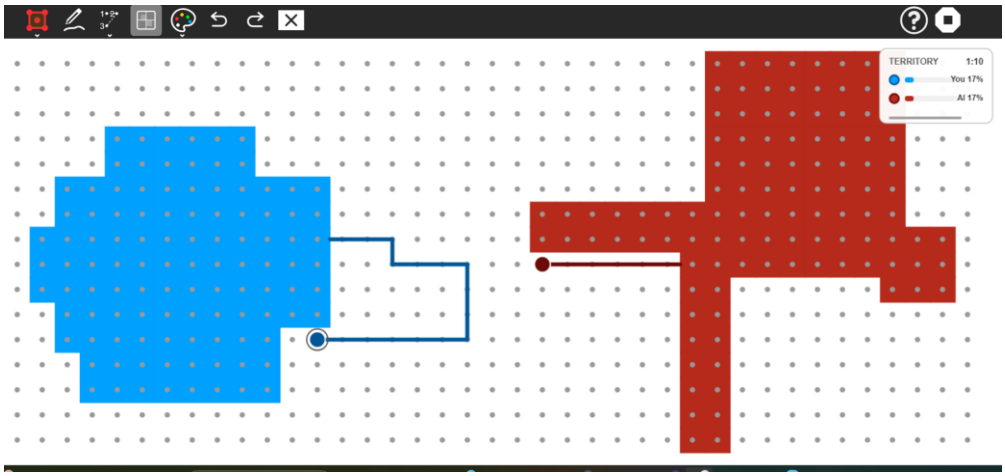
- Self-trail collision results in player elimination
- Crossing opponent trails cuts and removes them
- Head-to-head and boundary collisions are resolved with defined rules

❖ AI Behavior

- AI follows a simple **state-based strategy**:
  - Expands territory outward
  - Returns safely to close loops
  - Attempts to intercept opponent trails when possible

❖ Rendering and Feedback

- Displays player territories, trails, and movement in real time
- Scoreboard shows captured area and remaining time
- Game start and end states are clearly indicated



TIMELINE

Availability

- Fully available throughout the GSoC period
- No planned off-grid time

The implementation will build upon my existing prototype of the Connect The Dots activity, allowing the timeline to progress from an already functional baseline rather than starting from scratch.

Pre-GSOC Phase	Gain a thorough understanding of the Sugarizer codebase, identify issues and areas for improvement, and actively contribute by fixing issues.
Community Bonding Period	Discuss and finalize the design, modes, behavior (including collaboration), and implementation approach for the Connect The Dots activity with mentors.
Week 1	• Audit current functionality across Draw, Number and Capture modes (including undo/redo, save/load, and sharing) • Identify stable vs. fragile components and document gaps • Define final project scope with mentor (must-have, should-have, stretch goals) • Establish a QA workflow with test cases and bug tracking
Week 2	• Refine and stabilize mode transitions to preserve user progress • Improve undo/redo consistency across edge cases • Validate state serialization/deserialization to ensure reliable save/load
Week 3	• Improve built-in template loading and rendering consistency • Strengthen custom template creation and validation • Add metadata support (category, difficulty) for better organization
Week 4	• Improve sequence validation and guided progression logic • Add assistive feedback (step hints, incorrect input handling) • Optimize rendering performance for larger templates • Expand Number Mode by adding more templates (shapes, letters, numbers)
Week 5	• Improve shared session synchronization and reduce state mismatches • Ensure compatibility of persistence with updated structures • Prepare Evaluation 1 deliverables (demo, documentation, known issues)
Evaluation 1 (end of Week 5)	• Stable gameplay across all modes • Reliable save/load, undo/redo and template handling • Demo-ready implementation with documented progress
Week 6	• Improve interaction clarity (tool states, transitions, feedback messages) • Enhance accessibility (touch interaction, readability, usability) • Improve responsiveness across screen sizes and devices (mobile/tablet/desktop) • Incorporate initial mentor feedback
Week 7	• Implement high-priority features based on mentor feedback • Ensure compatibility with existing systems (templates, persistence, collaboration) • Add debugging/logging tools for easier issue tracking

Week 8	• Address smaller bugs and polish existing features • Improve documentation for current functionality • Ensure consistent behavior across all modes
Week 9	• Optimize rendering and interaction performance • Fix high-priority bugs from testing • Validate edge cases (rapid interactions, mode switching, persistence)
Week 10	• Finalize all core features and freeze major changes • Conduct full testing across devices and environments • Prepare Evaluation 2 deliverables (demo, QA report, known issues)
Evaluation 2	• Feature-complete and stable implementation • Major bugs resolved and workflows validated • Ready for final polishing and submission
Week 11	• Refine UI/UX details based on mentor feedback • Complete user and developer documentation • Perform regression testing across all features
Week 12	• Conduct final testing and fix critical issues • Prepare final demo and submission materials • Complete mentor review and outline post-GSoC plans

Remaining time will be utilized for adding additional features, incorporating mentor feedback, and improving documentation and overall quality

TIME COMMITTMENT

During the GSoC period, I will be able to devote approximately **10-12 hours per day**, ensuring steady progress.

Following this timeline, I am confident that I will complete and deliver a well-tested implementation before the final deadline (3rd Week of August). The plan includes buffer time for testing, feedback, and refinements, allowing flexibility without impacting the schedule

PROGRESS REPORTING

- I will conduct weekly mentor syncs to discuss progress, blockers, and next steps.
- I will share weekly written updates outlining planned vs completed work.
- I will follow a PR-based workflow with small, reviewable changes.
- I will provide additional updates if any blockers arise.
- I will share demo clips or screenshots every two weeks to showcase progress.

POST-GSOC PLANS

I plan to continue contributing to Sugar Labs after GSoC by:

- Maintaining and improving the Connect The Dots activity.
- Incorporating feedback from users and mentors.
- Contributing to documentation and onboarding resources.
- Assisting with testing and improvements for related activities.

Looking forward to contributing to Sugarizer this summer.

Kind regards,

Cheryl Pinto